

Documentación técnica de PoliTechNews

Santiago Calvo Patiño

Politécnico Grancolombiano

Front End

John Olarte Ramos

Medellín

26 de septiembre de 2026

Resumen

PoliTechNews es un prototipo de periódico digital universitario construido con HTML, CSS y JavaScript sin dependencias de ejecución. Esta documentación explica su arquitectura lógica, la distribución del repositorio, las responsabilidades de cada archivo y el funcionamiento de las rutinas que cargan noticias desde JSON, renderizan vistas, validan formularios y conservan favoritos y cambios editoriales en localStorage. El sistema utiliza seis documentos HTML como puntos de entrada, una hoja de estilos compartida, un módulo JavaScript autocontenido y un archivo JSON con los datos iniciales. La interfaz se adapta a escritorio, tableta y teléfono mediante cuadrículas, flexbox y puntos de ruptura. Las pruebas de humo automatizadas recorren el catálogo, la búsqueda, los filtros, el detalle, los favoritos, la gestión de noticias, el contacto y la navegación móvil. La arquitectura es adecuada para una primera versión académica del lado del cliente, pero no reemplaza un backend: los datos creados existen solo en el navegador y la validación de contacto no envía información a un servidor. El documento también identifica restricciones, medidas de seguridad aplicadas y una ruta de evolución hacia Angular y servicios persistentes.

Palabras clave: arquitectura front end, JavaScript, JSON, localStorage, validación, diseño responsivo

Tabla de contenido

Resumen.....	2
Tabla de contenido	3
Introducción	5
Objetivos de la documentación	5
Alcance y límites.....	5
Tecnologías y decisiones de diseño	6
Arquitectura del sistema	6
Capa de presentación	7
Capa de aplicación	7
Capa de datos y persistencia	8
Flujo de ejecución	8
Distribución del repositorio	8
Documentos HTML	10
Hoja de estilos.....	11
Diseño adaptable	11
Módulo JavaScript	12
Catálogo, búsqueda y paginación	13
Favoritos	14
Gestión básica de noticias	14

Eventos e inicialización	14
Modelo de datos JSON	15
Persistencia local.....	16
Validaciones y seguridad	16
Accesibilidad y experiencia de usuario.....	17
Pruebas y verificación.....	18
Ejecución y mantenimiento.....	18
Ejecución local.....	18
Repositorio y control de versiones.....	19
Convenciones de mantenimiento	19
Limitaciones y evolución propuesta	19
Conclusiones	20
Referencias.....	21

Introducción

Esta documentación técnica describe cómo está construida la segunda entrega de PoliTechNews y cómo colaboran sus componentes. Su finalidad es facilitar la evaluación académica, el mantenimiento del código y una futura migración a Angular. La explicación se basa en el código publicado en el repositorio del proyecto y en las orientaciones del módulo Front End.

El prototipo se ejecuta completamente en el navegador. Las noticias iniciales se solicitan con la API Fetch y se interpretan como JSON; las preferencias y modificaciones del usuario se serializan en localStorage. Fetch ofrece una interfaz basada en promesas para solicitar recursos y procesar sus respuestas (Mozilla, s. f.-a), mientras que localStorage conserva datos entre sesiones para un mismo origen (Mozilla, s. f.-b). Estas capacidades permiten demostrar interacción sin implementar todavía un servidor.

Objetivos de la documentación

1. Presentar la arquitectura y el flujo de ejecución del prototipo.
2. Explicar la distribución del repositorio y la responsabilidad de cada archivo.
3. Describir las funciones, estructuras de datos, eventos y validaciones del código.
4. Registrar las pruebas, restricciones técnicas y criterios de mantenimiento.

Alcance y límites

El documento cubre la versión estática disponible en la rama main del repositorio. No describe una API, base de datos, autenticación ni envío real de correo porque esas capacidades no forman parte de esta entrega. El mini CRUD y los favoritos son persistentes solo en el

navegador que los creó. La documentación distingue esta limitación para evitar interpretar el prototipo como una aplicación de producción.

Tecnologías y decisiones de diseño

Tabla 1
Tecnologías empleadas

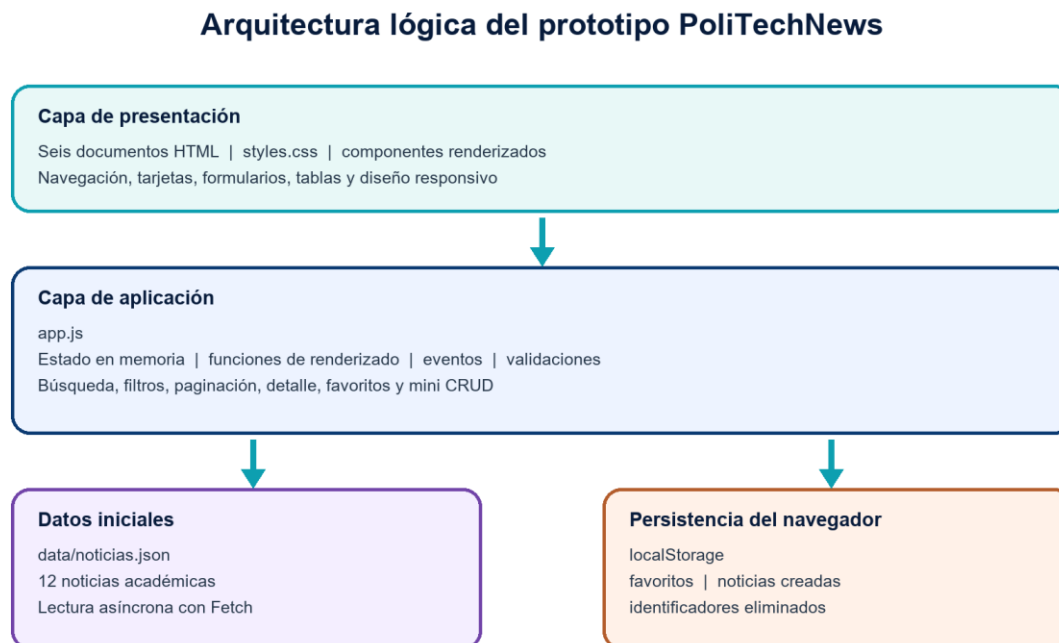
Tecnología	Uso en el proyecto	Decisión técnica
HTML5	Seis puntos de entrada semánticos	Mantiene rutas legibles y metadatos propios por vista.
CSS3	Diseño visual y responsivo	Centraliza tokens, componentes y tres rangos de adaptación.
JavaScript ES6+	Estado, renderizado e interacción	Evita dependencias y hace explícitos los fundamentos del módulo.
JSON	Catálogo inicial de 12 noticias	Separa el contenido de la presentación y permite carga dinámica.
Web Storage	Favoritos, altas y eliminaciones	Ofrece persistencia local sin backend (Mozilla, s. f.-b).
Playwright	Pruebas de humo	Automatiza recorridos críticos en un navegador real.
Git y GitHub	Control de versiones y publicación del código	Conserva historial y facilita revisión (GitHub, s. f.).

Nota. El prototipo no utiliza Bootstrap, Tailwind ni un framework JavaScript; la interfaz se implementa con capacidades nativas del navegador.

Arquitectura del sistema

PoliTechNews adopta una arquitectura front end por capas. Los documentos HTML y CSS conforman la presentación; app.js concentra el estado y la lógica de aplicación; noticias.json proporciona los registros iniciales; localStorage conserva las modificaciones del usuario. La separación es conceptual, pues todo se ejecuta en el mismo proceso del navegador, pero permite identificar responsabilidades y preparar una migración posterior.

Figura 1
Arquitectura lógica de PoliTechNews



Nota. Elaboración propia a partir de la implementación de PoliTechNews.

Capa de presentación

Los archivos HTML declaran idioma, codificación, metadatos, título, hoja de estilos y script. Cada body contiene un atributo data-page y un contenedor #app. La aplicación lee ese atributo para decidir qué vista debe renderizar. styles.css define el sistema visual, los componentes compartidos y los puntos de ruptura.

Capa de aplicación

app.js se encapsula en una función inmediatamente invocada. Esto evita introducir nombres en el espacio global. El módulo reúne constantes, estado, utilidades, constructores de interfaz, funciones de vista, acciones del usuario e inicialización. La delegación de eventos

permite que elementos generados después de la carga inicial respondan sin registrar un controlador individual para cada tarjeta o fila.

Capa de datos y persistencia

data/noticias.json actúa como fuente de solo lectura. El estado efectivo combina ese catálogo con las noticias creadas en el navegador y excluye los identificadores eliminados. Tres claves versionadas de localStorage separan favoritos, contenido personalizado y eliminaciones. Como los valores de Web Storage son cadenas, el módulo aplica JSON.stringify y JSON.parse al guardar y recuperar listas.

Flujo de ejecución

1. El navegador analiza el HTML y carga styles.css y app.js con defer.
2. init lee las tres colecciones persistidas y muestra un indicador de carga.
3. fetch solicita data/noticias.json; el código verifica el estado HTTP y el tipo del resultado.
4. El módulo enlaza los eventos globales y selecciona el renderizador según data-page.
5. Cada acción actualiza state, localStorage o la URL, y vuelve a renderizar la región afectada.
6. Si la carga falla, se muestra una instrucción para ejecutar el proyecto mediante un servidor local.

Distribución del repositorio

```
PoliTechNews/  
├─ index.html, noticias.html, detalle.html  
├─ favoritos.html, gestion.html, contacto.html  
├─ app.js  
├─ styles.css  
└─ data/noticias.json
```

```

├── assets/images/
├── tests/smoke.js
├── output/docx/ y output/pdf/
├── README.md
└── build_*.py

```

La raíz contiene los archivos necesarios para ejecutar el sitio y revisar su entrega. Los recursos se agrupan por tipo. La carpeta output contiene artefactos académicos; tests contiene la verificación automatizada; los scripts build generan documentación y no intervienen durante la ejecución de la aplicación.

Tabla 2
Responsabilidad de los archivos principales

Archivo o ruta	Responsabilidad	Dependencias directas
index.html	Punto de entrada de Inicio.	styles.css y app.js
noticias.html	Punto de entrada del catálogo.	styles.css, app.js y noticias.json
detalle.html	Punto de entrada del detalle por parámetro id.	app.js, JSON y query string
favoritos.html	Lista personalizada de publicaciones guardadas.	app.js y localStorage
gestion.html	Formulario editorial, estadísticas y mini CRUD.	app.js, FileReader y localStorage
contacto.html	Formulario validado y preguntas frecuentes.	app.js y validación HTML
app.js	Estado, carga de datos, plantillas, eventos y persistencia.	DOM, Fetch, History, Storage y FileReader
styles.css	Tokens, componentes, vistas y reglas responsivas.	Fuentes web con alternativas locales
data/noticias.json	Catálogo base estructurado.	Consumido por fetch en init
assets/images/	Logo, portada e ilustraciones por categoría.	Referenciadas por HTML, JS y JSON
tests/smoke.js	Prueba automatizada de recorridos críticos.	Playwright y Chrome
README.md	Ejecución, alcance y criterios de revisión.	Repositorio GitHub

Documentos HTML

Los seis documentos comparten una plantilla mínima. Esta decisión evita duplicar cabecera, menú, pie y contenido. app.js genera esos componentes una vez y los adapta a la vista actual.

```
<body data-page="noticias">
  <div id="app"></div>
</body>
```

data-page funciona como un identificador de ruta. #app es el nodo de montaje. El script se carga con defer para ejecutarse cuando el documento ya fue analizado, y cada archivo conserva un title y una meta description específicos. Esta estructura facilita el SEO básico, la navegación sin framework y la lectura del código.

Tabla 3
Correspondencia entre documentos y renderizadores

Valor data page	Archivo	Función de vista	Resultado
inicio	index.html	renderHome	Portada, destacados, categorías y CTA.
noticias	noticias.html	renderNews	Buscador, filtros, tarjetas y paginación.
detalle	detalle.html	renderDetail	Artículo, favoritos, relacionados y compartir.
favoritos	favoritos.html	renderFavorites	Selección persistida y ordenamiento.
gestion	gestion.html	renderAdmin	Formulario, estadísticas, filtros y eliminación.
contacto	contacto.html	renderContact	Información, formulario y preguntas frecuentes.

Hoja de estilos

styles.css contiene el sistema visual completo. Las variables declaradas en :root almacenan colores, sombra y radios; así se evita repetir valores y se mantiene correspondencia con los mockups. Los componentes comunes preceden a los estilos de cada vista para que la cascada avance de lo general a lo específico.

Tabla 4
Organización de styles.css

Bloque		Selectores representativos	Función
Base		:root, *, body, h1-h3, input	Reinicio, tokens, tipografía y controles.
Estructura		.container, .section	Ancho máximo y espaciado vertical.
Navegación		.site-header, .main-nav, .menu-toggle	Menú de escritorio y móvil.
Componentes		.button, .toast, .banner, .panel	Acciones, mensajes y contenedores reutilizables.
Inicio	card	.home-hero, .news-grid, .community-	Portada y tarjetas destacadas.
Detalle	aside	.article-page, .article-layout, .article-	Lectura principal y contenido relacionado.
Favoritos		.favorite-card, .toolbar, .tip-panel	Listado personal y acciones.
Gestión		.admin-grid, .stats-grid, .admin-table	Panel editorial y tabla de noticias.
Contacto		.contact-grid, .contact-info, .faq-grid	Datos de contacto, formulario y FAQ.
Responsividad		@media 1020, 760 y 560 px	Reorganiza cuadrículas, menú, tablas y formularios.

Diseño adaptable

A 1020 píxeles se oculta el CTA del encabezado y los paneles complejos pasan a una columna. A 760 píxeles aparece el menú móvil, el hero omite la ilustración decorativa y las cuadrículas reducen columnas. A 560 píxeles las tarjetas, formularios y acciones se apilan. Las pruebas verifican que una ventana de 390 píxeles no produzca desplazamiento horizontal.

Módulo JavaScript

app.js contiene 33 funciones agrupables en siete responsabilidades. La variable state mantiene el modelo actual de la interfaz: datos base, registros personalizados, elementos eliminados, favoritos, filtros y páginas. Las constantes CATEGORIES, DEFAULT_IMAGE y KEY centralizan opciones y nombres de almacenamiento.

Tabla 5
Funciones de almacenamiento, datos y utilidades

Función	Entrada y salida	Responsabilidad
read	clave y valor alternativo -> arreglo	Lee JSON de localStorage; ante bloqueo o formato inválido devuelve el valor seguro.
save	clave y valor -> booleano	Serializa una lista; informa al usuario si el navegador no puede guardarla.
esc	cualquier valor -> texto HTML seguro	Escapa &, <, >, comillas dobles y simples antes de insertar contenido editable.
plain	valor -> cadena	Convierte a texto y elimina espacios exteriores.
dateLabel	fecha ISO -> fecha es CO	Formatea fechas con Intl.DateTimeFormat y zona UTC.
allArticles	estado -> arreglo	Fusiona registros base y personalizados y descarta los identificadores eliminados.
published	estado -> arreglo	Conserva solo publicados y ordena por fecha descendente.
getArticle	id -> noticia o undefined	Busca una noticia dentro del catálogo efectivo.
href	id -> URL	Construye detalle.html?id=... con codificación segura.
categoryClass	categoría -> clase CSS	Asocia categorías con etiquetas visuales.
imageSrc	noticia -> ruta o data URL	Acepta recursos locales o imágenes cargadas; usa una alternativa si el origen no es válido.

Tabla 6
Funciones de componentes y renderizado

Función	Responsabilidad
nav	Construye marca, navegación, estado activo, botón móvil y CTA.

Función	Responsabilidad
footer	Genera información institucional, enlaces y región aria live para avisos.
shell	Monta cabecera, main y pie dentro de #app.
announce	Muestra un toast temporal de éxito o error.
banner	Crea la cabecera reutilizable de las vistas internas.
card	Convierte una noticia en tarjeta con imagen, categoría, resumen y enlace.
emptyState	Presenta un estado vacío consistente y una acción opcional.
renderHome	Compone hero, tres noticias recientes, categorías y bloque comunitario.
renderNews	Monta filtros y entrega el listado a newsResults.
renderDetail	Lee id, valida disponibilidad, muestra contenido, favoritos y relacionados.
renderFavorites	Resuelve identificadores guardados, ordena y presenta la colección.
renderAdmin	Calcula estadísticas y construye formulario y panel de gestión.
renderContact	Construye información, formulario validado y preguntas frecuentes.

Catálogo, búsqueda y paginación

filteredNews normaliza la consulta a minúsculas con configuración de español y aplica dos condiciones: coincidencia exacta de categoría y presencia del texto en título, resumen o categoría. pagination construye controles con estado activo, botones anterior y siguiente y atributos aria. newsResults calcula el número de páginas, limita el índice actual, recorta seis noticias y devuelve tarjetas o un estado vacío.

```
const items = filteredNews();
const perPage = 6;
const pages = Math.max(1, Math.ceil(items.length / perPage));
state.page = Math.min(state.page, pages);
const shown = items.slice((state.page - 1) * perPage, state.page * perPage);
```

Este cálculo evita páginas inexistentes después de cambiar un filtro. Al seleccionar una categoría, history.replaceState sincroniza la URL sin recargar el documento.

Favoritos

`toggleFavorite` alterna el identificador de una noticia dentro de `state.favorites`, persiste la lista y actualiza la vista correspondiente. Guardar solo identificadores evita duplicar el objeto completo. Cuando se abre Favoritos, `renderFavorites` resuelve cada id contra el catálogo efectivo; por ello, una noticia eliminada deja de mostrarse aunque su id estuviera guardado.

Gestión básica de noticias

`imageForForm` valida que una imagen sea PNG, JPEG o WebP y no supere 1 MB. `FileReader` la convierte en data URL para almacenarla junto al registro. Si el usuario no carga una imagen, se reutiliza la ilustración de la categoría. `createArticle` ejecuta `reportValidity`, lee el formulario con `FormData`, divide el contenido por párrafos, calcula un tiempo de lectura aproximado y guarda un registro `published` o `draft`.

```
const article = {
  id: `user-${Date.now()}-${Math.random().toString(36).slice(2, 7)}`,
  title: plain(fields.get("title")),
  category: plain(fields.get("category")),
  content,
  status: mode === "draft" ? "draft" : "published"
};
```

La eliminación distingue registros personalizados y noticias base. Los primeros se retiran de la colección `custom`; los segundos conservan el JSON intacto y agregan su id a `deleted`.

También se elimina el id de favoritos para evitar referencias huérfanas.

Eventos e inicialización

Tabla 7

Funciones de interacción y ciclo de vida

Función	Funcionamiento
<code>onClick</code>	Delega categorías, paginación, limpieza de filtros, favoritos, eliminación, compartir y navegación administrativa mediante <code>data-action</code> .

Función	Funcionamiento
onSubmit	Intercepta búsqueda, filtros administrativos, creación editorial y contacto; impide recargas innecesarias.
bindEvents	Registra click, submit, change e input una sola vez; también controla el menú móvil y el contador de caracteres.
init	Recupera Storage, solicita el JSON, valida el resultado, enlaza eventos y despacha la función de vista.

El objeto que despacha vistas funciona como un enrutador liviano: asigna a cada valor de data-page su renderizador. Si el valor no existe, renderHome opera como alternativa segura.

Modelo de datos JSON

noticias.json contiene 12 objetos con una estructura uniforme. La separación entre datos y vista permite reutilizar la misma información en Inicio, Noticias, Detalle, Favoritos y Gestión.

Tabla 8
Esquema de una noticia

Propiedad	Tipo	Uso
id	string	Identificador único usado en URL, favoritos y eliminación.
title	string	Título de tarjeta, detalle y búsqueda.
category	string	Filtro, etiqueta y selección de imagen.
summary	string	Resumen de tarjetas y panel lateral.
date	string ISO	Orden cronológico y etiqueta localizada.
minutes	number	Tiempo estimado de lectura.
image	string	Ruta de recurso o data URL validada.
author	string	Crédito editorial.
content	string[]	Párrafos del cuerpo de la noticia.
status	opcional string	published o draft en noticias creadas localmente.

Persistencia local

Tabla 9
Claves de localStorage

Clave	Contenido	Actualización
politechnews:favorites:v1	Arreglo de identificadores favoritos.	toggleFavorite y eliminación.
politechnews:custom:v1	Objetos creados o borradores.	createArticle y eliminación.
politechnews:deleted:v1	Identificadores base ocultos.	Eliminación desde Gestión.

El sufijo v1 permite cambiar el formato en una versión futura sin sobrescribir silenciosamente datos antiguos. read captura errores de análisis o acceso y devuelve un arreglo seguro. save captura errores de cuota o permisos y muestra un mensaje. De acuerdo con MDN, localStorage conserva datos entre sesiones pero depende del origen; por eso el proyecto debe ejecutarse con un servidor local y no mediante file:// (Mozilla, s. f.-b).

Validaciones y seguridad

Los formularios combinan restricciones HTML con reportValidity. HTML define required, type=email, minlength, maxlength y límites de archivos; JavaScript controla el flujo y presenta confirmación. La validación del cliente mejora la experiencia, pero no sustituye una validación de servidor cuando existen datos remotos (Mozilla, s. f.-c). En PoliTechNews el contacto es simulado y no transmite información.

Tabla 10
Controles de seguridad y robustez

Control	Implementación	Riesgo mitigado
---------	----------------	-----------------

Control	Implementación	Riesgo mitigado
Escape HTML	esc procesa contenido antes de usar innerHTML.	Inyección de etiquetas desde entradas editables.
Lista permitida de imágenes	imageSrc acepta assets/images o data:image.	Carga arbitraria de orígenes no previstos.
Tipo y tamaño de archivo	PNG, JPEG o WebP hasta 1 MB.	Datos incompatibles y consumo excesivo de Storage.
Validación de JSON	init exige response.ok y un arreglo.	Renderizado con respuesta inválida.
Confirmación de borrado	confirm antes de retirar contenido.	Eliminación accidental.
Manejo de errores	try/catch en Storage, archivo y carga inicial.	Fallas silenciosas.

Accesibilidad y experiencia de usuario

- El documento declara lang=es y títulos específicos por página.
- El enlace Saltar al contenido permite omitir la navegación con teclado.
- aria-current identifica la sección activa y aria-expanded comunica el estado del menú.
- Los avisos usan role=status y aria-live=polite.
- Los botones de paginación y favoritos tienen etiquetas accesibles y estados disabled o aria-pressed.
- Las imágenes de contenido incluyen texto alternativo; los adornos se marcan como ocultos para tecnologías de asistencia.

La implementación aplica una base accesible, pero una entrega de producción debería incorporar auditorías con lectores de pantalla, contraste automatizado y navegación completa por teclado.

Pruebas y verificación

tests/smoke.js inicia Chrome mediante Playwright, crea un contexto aislado y registra errores de página. Las aserciones verifican tanto el número de elementos como la persistencia después de recargar. Las capturas generadas por la prueba sirven para revisión visual, pero no se almacenan en Git porque tmp está ignorado.

Tabla 11
Cobertura de la prueba de humo

Recorrido	Comprobación
Inicio	Tres tarjetas destacadas y captura completa.
Noticias	Seis resultados por página, filtro Cloud y búsqueda por texto.
Detalle y favoritos	Alta de favorito, visualización y persistencia tras recargar.
Gestión	Publicación, visibilidad en catálogo, eliminación y borrador no publicado.
Contacto	Campos válidos, privacidad y mensaje de confirmación.
Móvil	Menú abre y cierra; ancho de documento no supera 390 px.
Errores	La colección pageerror termina vacía.

```
$env:NODE_PATH='.../node_modules'
node tests/smoke.js

SMOKE_OK: inicio, catálogo, búsqueda, filtros, detalle,
favoritos, CRUD, contacto y móvil
```

Ejecución y mantenimiento

Ejecución local

```
cd C:\Users\santy\Documents\ChatGPT\webPage
python -m http.server 8000
# Abrir http://localhost:8000
```

Se requiere un servidor HTTP porque fetch debe solicitar el archivo JSON dentro de un origen válido. Abrir index.html directamente con file:// puede bloquear la solicitud.

Repositorio y control de versiones

El código y los documentos se encuentran en <https://github.com/santycp/PoliTechNews>.

Git conserva el historial de cambios y GitHub publica el repositorio para revisión. Un sistema de control de versiones permite rastrear qué cambió, quién lo hizo y cuándo ocurrió (GitHub, s. f.).

Convenciones de mantenimiento

- Conservar los id de noticias únicos y estables.
- Añadir categorías tanto a CATEGORIES como a categoryClass y a la selección de imagen.
- Actualizar el sufijo de claves si cambia el esquema persistido.
- Mantener las plantillas con esc para cualquier contenido que provenga de formularios.
- Ejecutar `node --check app.js` y `tests/smoke.js` antes de publicar cambios.
- No modificar archivos generados de output sin regenerar la documentación correspondiente.

Limitaciones y evolución propuesta

Tabla 12

Limitaciones y alternativa de evolución

Limitación actual	Efecto	Evolución recomendada
Sin backend	Los datos no se comparten entre dispositivos.	API REST y base de datos.
Sin autenticación	Gestión está disponible para cualquier visitante.	Roles y sesión segura.
localStorage	Cuota limitada y sin sincronización.	Persistencia remota; Storage solo para preferencias.
Plantillas HTML en cadenas	app.js concentra presentación y lógica.	Componentes Angular y servicios.

Limitación actual	Efecto	Evolución recomendada
Contacto simulado	No llega una solicitud al equipo.	Servicio de correo o endpoint de formularios.
Prueba de humo única	Cobertura parcial de casos extremos.	Pruebas unitarias, accesibilidad y CI.

La migración a Angular puede conservar el modelo de noticia, las rutas y la identidad visual. Las funciones `renderHome`, `renderNews` y demás se convertirían en componentes; `read` y `save` pasarían a un servicio de persistencia; el estado de filtros podría gestionarse mediante propiedades y binding; la carga JSON se realizaría con `HttpClient`. Esta transición reduce el tamaño del módulo único y coincide con la Entrega 3 indicada en las orientaciones.

Conclusiones

La arquitectura de `PoliTechNews` separa de forma comprensible la presentación, la lógica, los datos y la persistencia dentro de un prototipo del lado del cliente. Los seis archivos HTML actúan como rutas; `styles.css` garantiza una identidad visual compartida y adaptable; `app.js` coordina carga, renderizado, validaciones y eventos; JSON y `localStorage` proporcionan datos iniciales y continuidad local.

La explicación función por función demuestra que el sistema no consiste solo en páginas estáticas. La carga asíncrona, los filtros, la paginación, los favoritos, el mini CRUD y las validaciones responden a un mismo estado controlado. Las pruebas automatizadas respaldan los recorridos principales. Las limitaciones registradas son coherentes con una segunda entrega académica y definen una ruta clara hacia Angular, persistencia remota y despliegue en la siguiente fase.

Referencias

Calvo Patiño, S. (2026). PoliTechNews [Código fuente]. GitHub.

<https://github.com/santycp/PoliTechNews>

GitHub. (s. f.). Acerca de Git. GitHub Docs. <https://docs.github.com/es/get-started/using-git/about-git>

Mozilla. (s. f.-a). Uso de Fetch. MDN Web Docs.

https://developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch

Mozilla. (s. f.-b). Window.localStorage. MDN Web Docs.

<https://developer.mozilla.org/es/docs/Web/API/Window/localStorage>

Mozilla. (s. f.-c). Validación de formularios en el lado del cliente. MDN Web Docs.

https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Forms/Form_validation

Olarte Ramos, J. (2026). Orientaciones para las entregas del módulo Front End Agosto 2026-1 [Documento de curso]. Politécnico Grancolombiano.